

Tutorial 09 – Searching Algorithms

1. Write a Java program to implement the **linear search algorithm**.

Your program should:

- Accept the size of the array and its elements from the user.
- Ask the user to enter the target element to search.
- Use a method `linearSearch(int[] array, int target)` to perform the search.
- Print the index of the target element if found or display a message that the element is not found.

2. Write a Java program to implement **binary search** using an **iterative method**.

Your program should:

- Accept a sorted array of integers from the user.
- Ask the user to enter the target element to search.
- Use a method `binarySearch(int[] array, int target)` to perform the search iteratively.
- Print the index of the target element if found or display a message that the element is not found.

3. Write a Java program to implement **hash-based searching** using a **HashMap**.

Your program should:

- Allow the user to enter a list of key-value pairs (e.g., ID and name).
- Store the pairs in a `HashMap<Integer, String>`.
- Prompt the user to enter a key to search.
- Use a method `searchInHashMap(HashMap<Integer, String> map, int key)` to perform the search.
- Display the corresponding value if the key is found, or display a message saying the key was not found.

4. A teacher stores students' marks in the order they were entered into the system. Since the marks are not sorted, the teacher needs to search each mark one by one to find a specific student's mark.

Write a Java program to implement the **Linear Search algorithm**.

Your program should:

- Accept the number of student marks from the user.
- Accept the list of student marks.
- Ask the user to enter the mark to search.
- Use a method `linearSearch(int[] marks, int targetMark)` to perform the search.
- Print the index of the mark if found, or display "**Mark not found.**"

5. A mobile phone stores contact numbers in ascending order. To quickly locate a contact number, it uses the Binary Search algorithm.

Write a Java program to implement **Binary Search using an iterative method**.

Your program should:

- Accept the number of contact numbers.
- Accept the contact numbers in sorted order.
- Ask the user to enter the contact number to search.
- Use a method `binarySearch(int[] contacts, int targetContact)` to perform the search iteratively.
- Print the index of the contact number if found, or display "Contact number not found."

6. A bank stores customer account information using the account number as the key and the customer name as the value. The bank uses a HashMap to retrieve customer details quickly. Write a Java program to implement **hash-based searching** using a **HashMap**.

Your program should:

- Allow the user to enter a list of account number and customer name pairs.
- Store the pairs in a `HashMap<Integer, String>`.
- Ask the user to enter an account number to search.
- Use a method `searchAccount(HashMap<Integer, String> accounts, int accountNumber)` to perform the search.
- Display the customer name if the account number is found, or display "Account number not found."

7. A supermarket stores product IDs in the order they are added to the inventory. Since the product IDs are not sorted, the cashier searches each product one by one.

Write a Java program to implement the **Linear Search algorithm**.

Your program should:

- Accept the number of product IDs.
- Accept the list of product IDs.
- Ask the user to enter a product ID to search.
- Use a method `linearSearch(int[] products, int targetProductId)` to perform the search.
- Print the index of the product if found, or display **"Product not found."**

8. An airline stores flight numbers in ascending order for efficient searching.

Write a Java program to implement **Binary Search using an iterative method**.

Your program should:

- Accept the number of flight numbers.
- Accept the flight numbers in sorted order.
- Ask the user to enter a flight number to search.
- Use a method `binarySearch(int[] flights, int targetFlight)` to perform the search iteratively.
- Print the index of the flight number if found, or display **"Flight number not found."**

9. A vehicle registration department stores vehicle registration numbers as keys and owner names as values in a `HashMap`.

Write a Java program to implement **hash-based searching** using a **HashMap**.

Your program should:

- Allow the user to enter vehicle registration numbers and owner names.
- Store the pairs in a `HashMap<Integer, String>`.
- Prompt the user to enter a vehicle registration number to search.
- Use a method `searchVehicle(HashMap<Integer, String> vehicles, int registrationNumber)` to perform the search.
- Display the owner's name if the registration number is found, or display **"Vehicle registration number not found."**

10. A bus station stores bus route numbers in the order they were registered. Since the routes are not sorted, the system searches sequentially.

Write a Java program to implement the **Linear Search algorithm**.

Your program should:

- Accept the number of bus route numbers.
- Accept the list of bus route numbers.
- Ask the user to enter a route number to search.
- Use a method `linearSearch(int[] routes, int targetRoute)` to perform the search.
- Print the index of the route if found, or display "Route not found."

11. A hospital maintains patient IDs in ascending order to enable faster searching.

Write a Java program to implement **Binary Search using an iterative method**.

Your program should:

- Accept the number of patient IDs.
- Accept the patient IDs in sorted order.
- Ask the user to enter a patient ID to search.
- Use a method `binarySearch(int[] patientIds, int targetPatientId)` to perform the search iteratively.
- Print the index of the patient ID if found, or display "Patient ID not found."

12. A university stores student IDs as keys and student names as values using a HashMap to provide quick access to student records.

Write a Java program to implement **hash-based searching** using a **HashMap**.

Your program should:

- Allow the user to enter student ID and student name pairs.
- Store the pairs in a `HashMap<Integer, String>`.
- Prompt the user to enter a student ID to search.
- Use a method `searchStudent(HashMap<Integer, String> students, int studentId)` to perform the search.
- Display the student's name if the student ID is found, or display "Student ID not found."